

GenAI Take-Home Assignment

Smart Document Assistant

Target: College graduates / entry-level software & GenAI candidates

Maximum implementation time: 8 hours

Submission: Working application + source code + README + architecture diagram + 5–8 minute video

Important: You are free to use online resources and AI tools such as ChatGPT, Claude, Gemini, GitHub Copilot, Stack Overflow, GitHub repositories and official documentation. We evaluate your ability to understand, build, debug and explain a solution—not your ability to memorize syntax.

1. Objective

Build a small application that allows a user to upload one or more documents and ask questions about their contents using Generative AI. The assignment is designed to evaluate GenAI capability, coding skill, problem solving, creativity and communication.

2. Problem Statement

Build a **Smart Document Assistant** that allows a user to upload documents and ask questions about their contents. The application should retrieve relevant information from the uploaded documents and use an LLM to generate a useful answer.

Example:

A user uploads: *company_policy.pdf*, *employee_handbook.pdf* and *leave_policy.pdf*.

The user asks: “How many annual leaves does an employee get?”

The system should answer using the uploaded documents and, preferably, identify the source document/section used for the answer.

3. Minimum Functional Requirements

A. Document Upload

- Allow the user to upload at least PDF and TXT documents.
- DOCX, CSV or Excel support is optional.

B. Document Processing

- Extract text from uploaded documents.
- Split content into searchable chunks or another appropriate representation.
- Create embeddings or another semantic/search representation.
- Store the information in a searchable store.
- Reasonable choices include FAISS, Chroma, Pinecone, Weaviate, Elasticsearch, pgvector, SQLite or another suitable approach.

C. Question Answering

- Provide a question input for the user.
- Retrieve relevant information from the uploaded documents.
- Use an LLM to generate the answer.
- Keep the answer grounded in retrieved document content.

D. Source / Citation

- Preferably show the document or source section used to generate the answer.
- Exact page-level citations are desirable but not mandatory.

4. Hallucination / Unknown-Question Handling

This is an important requirement. If the uploaded documents do not contain the information needed to answer a question, the application should not confidently invent an answer.

Example: If the documents contain leave policy, insurance and working-hours information, and the user asks “What is the company’s stock price?”, a reasonable response is: *“I couldn’t find information about the company’s stock price in the uploaded documents.”*

Explain in your README how your solution attempts to reduce hallucination.

5. User Interface

The UI does not need to be highly polished. A simple Streamlit, Gradio, React, HTML/JS or similar interface is sufficient.

Suggested flow: Upload Documents → Show Uploaded Documents → Ask Question → Show Answer → Show Sources

6. Creativity Requirement

Implement at least **one additional feature** beyond the minimum requirements. You decide what to build.

- Conversation memory / follow-up questions
- Document comparison
- Answer confidence or evidence indicator
- Automatic document summary
- Query suggestions
- Multi-document reasoning
- A useful feature of your own design

Do not add features simply for quantity. A small, well-designed feature that solves a real user problem is valuable.

7. Technical Expectations

Provide a simple architecture diagram showing the main components and data flow.

A typical flow could be: User → UI → Backend → Document Parser → Chunking → Embeddings/Search Store → Retrieval → LLM → Response.

No specific programming language, framework, LLM, embedding model or vector database is mandatory. Choose technologies you can explain.

8. README Requirements

- Problem understanding — explain the problem in your own words.
- Architecture — explain components and data flow.
- Technology choices — explain why you selected the main technologies.
- How to run — include setup commands and environment variables.
- AI tools used — mention where ChatGPT, Claude, Gemini, Copilot or other tools helped.
- Known limitations — state what your solution does not handle well.

Never commit API keys, passwords or other secrets.

9. Short Video Explanation

Length: 5–8 minutes. The video should demonstrate the working application and explain the key technical decisions.

- **1 minute:** Explain the problem and your approach.
- **2–3 minutes:** Demo the application — upload documents, ask questions, show an answer and source, ask an unanswerable question, and demonstrate your creative feature.
- **1–2 minutes:** Explain the architecture and data flow.
- **1 minute:** Explain important technical decisions and how you handled hallucinations.
- **30–60 seconds:** Explain what you would improve with another two days.

10. Submission Package

- GitHub repository or ZIP containing the source code.
- README.md with setup and design decisions.
- Architecture diagram.
- 5–8 minute demo/explanation video.
- Any sample documents required to run the application.
- A short note estimating how the 8 hours were spent.

11. Time Constraint

Do not spend more than 8 hours. This is not intended to be a production-ready system. We are primarily interested in how you approach an ambiguous problem, make technical decisions, use GenAI tools, solve problems, test your work and communicate your solution.

Suggested time log: Problem understanding 30 min | Development 5 hr | Testing/debugging 1.5 hr | Documentation/video 1 hr

12. What We Will Look For

Area	What we are looking for
GenAI capability	LLM integration, prompting, retrieval/RAG thinking, grounding, hallucination handling, source attribution
Coding skill	Readable structure, modularity, error handling, API/UI implementation, configuration and security basics
Problem solving	Problem decomposition, edge cases, debugging, trade-offs and handling ambiguity
Creativity	Useful additional feature, UX/product thinking and thoughtful improvements
Communication	Ability to clearly explain architecture, decisions, limitations and future improvements

13. Candidate Guidance

Use the tools available to you. We do not expect you to know every library or API before starting. What matters is that you can make sensible decisions, verify what AI-generated code does, and explain your implementation.

Think like a builder: understand → decompose → design → build → test → improve → explain.

Good luck!